Toward Quantum-Resilient Decentralized Execution: Architectural Design, Deterministic Precompiles, and Empirical Evaluation of a Sovereign Falcon-512 EVM Blockchain

Bashar Khan[a,*], Zaheer Khan[a]

[a]Department of Computer Science and Engineering, QR-DLT Research Lab

ARTICLE INFO

Keywords:
Post-Quantum Cryptography
Ethereum Virtual Machine (EVM)
Falcon-512 (Draft FIPS 206)
Segregated Witness
Native Precompiles
Account Abstraction (ERC-4337)
Empirical Benchmarking

ABSTRACT

Decentralized state machines powered by the Ethereum Virtual Machine (EVM) secure over a trillion dollars in digital assets using Elliptic Curve Digital Signature Algorithms (ECDSA secp256k1). However, Shor's polynomial-time discrete logarithm quantum algorithm threatens the foundational assumption of elliptic curve cryptography. Recent resource estimations by Babbush et al. (Google Quantum AI, 2026) calculate that breaking 256-bit elliptic curves requires only 1,200–1,450 logical qubits executing in under 10 minutes on fewer than half a million physical qubits. While post-quantum cryptography has matured through the NIST standardization of lattice-based schemes (FIPS 204 ML-DSA and draft FIPS 206 FN-DSA/Falcon), porting lattice cryptography to smart contract execution environments exposes what we formulate as the *Post-Quantum Blockchain Trilemma*: the fundamental trade-off between (1) post-quantum lattice hardness (NIST Security Level 1), (2) deterministic, low-cost EVM execution gas parity, and (3) network bandwidth and storage scalability under the 15.2× wire "Lattice Tax" of oversized public keys and signatures.

In this paper, we present the design, formalization, and empirical evaluation of **Falcon-EVM**: a sovereign, post-quantum EVM blockchain running native Falcon-512 cryptographic identity. Falcon-EVM introduces an EIP-2718 Type `0x46` typed transaction envelope, standard 20-byte address derivation (Address = Keccak-256(pk)[12..31]), and a native EVM precompile at address `0x15` executing integer-only AVX2 deterministic polynomial verification calibrated at exactly 3,000 gas (CPU execution parity with ecrecover). To defeat storage exhaustion, Falcon-EVM incorporates a Dual-Root Segregated Witness (SegWit) consensus architecture that decouples transaction execution intents from 1.56 KB lattice witnesses, enabling epoch-based witness pruning after confirmation. We demonstrate Turing-complete smart contract execution via deployed ERC-20 tokens, constant-product AMM DEXs ($x \cdot y = k$), and post-quantum ERC-4337 Account Abstraction wallets. In multi-trial empirical benchmarking ($N = 50$, 95% CI) against a normal EVM baseline (secp256k1), Falcon-EVM achieves 61,332 **sigs/sec** parallel verification throughput (1.22 ± 0.04× faster than ECDSA), reclaims 93.6% of historical disk storage via SegWit pruning (1,526.37 KB purged per 1,000 txs), matches classical propagation latency on historical blocks, and eliminates vulnerability to Shor's algorithm by rooting identity in NTRU lattice shortest-vector hardness.

1. Problem Formulation & The Post-Quantum Trilemma

1.1. The Quantum Threat to Decentralized State Machines

Decentralized blockchains and smart contract platforms constitute the trust layer of modern programmable finance. At the foundational core of the Ethereum Virtual Machine (EVM) is the assumption that public-key cryptography provides asymmetric computational security: signing an authorization requires private key sk, whereas verification requires only public key pk.

In classical EVM architectures (Wood, 2014), account authentication relies exclusively on the Elliptic Curve Digital Signature Algorithm (ECDSA) over the Koblitz curve secp256k1. Addresses are defined as the rightmost 20 bytes of the Keccak-256 hash of the uncompressed 64-byte public key:

$$\text{Address} = \text{Keccak-256}(pk_x \parallel pk_y)[12..31] \tag{1}$$

While addresses obscure public keys prior to the first outbound transaction, the very act of broadcasting a signed transaction to the peer-to-peer mempool permanently exposes (r, s, v), from which the full elliptic curve public key $Q \in E(\mathbb{F}_p)$ is publicly recovered via the ecrecover primitive:

$$Q = \text{ecrecover}(h, v, r, s) \tag{2}$$

In 1994, Peter Shor formulated a quantum polynomial-time algorithm for prime factorization and discrete logarithms (Shor, 1994). In 2003, Proos and Zalka established that computing elliptic curve discrete logarithms (ECDLP) over prime fields $\mathbb{F}_p$ is significantly easier for quantum computers than factoring RSA moduli of equivalent classical security (Proos and Zalka, 2003). Modernized resource estimations by Babbush et al. (Babbush et al., 2026) estimate that breaking 256-bit elliptic curves requires only 1,200 to 1,450 logical qubits executing in under 10 minutes on fewer than 5×10^5 physical qubits under surface-code architectures.

In a distributed ledger context, Aggarwal et al. demonstrated that quantum adversaries can weaponize Shor's algorithm through mempool front-running (Aggarwal et al.,

*Corresponding author
bashar@qrdlt.io (B. Khan); zaheer@qrdlt.io (Z. Khan)
ORCID(s):

2018). While Ethereum's target slot time is 12 seconds, periods of severe network congestion expand transaction queuing delays. A quantum adversary observing a mempool broadcast can solve for the private key $sk = \text{ECDLP}(Q)$, construct a replacement transaction with an elevated priority fee (gasPrice), and drain the account. Crucially, any address that has ever sent an outbound transaction has its public key permanently exposed on-chain at rest, making classical accounts immediately vulnerable to offline key extraction.

1.2. Genesis: From Generic Ledgers to EVM-Native Execution

Initial post-quantum distributed ledger research concentrated almost exclusively on unspent transaction output (UTXO) architectures (Aggarwal et al., 2018). While UTXO designs (such as Bitcoin) can isolate one-time signatures by enforcing strict single-use addresses, they cannot support stateful, Turing-complete decentralized applications.

Modern decentralized financial ecosystems (automated market makers, collateralized lending protocols, decentralized identity registries) depend on persistent smart contracts. A smart contract address is permanent; its storage trie, token balances, and interface logic are reused across millions of transactions. Enforcing single-use public keys in smart contracts is fundamentally incompatible with stateful programming.

Furthermore, naive attempts to integrate post-quantum verification into EVM chains by compiling lattice verification routines directly into interpreted EVM bytecode (Solidity/Vyper) fail catastrophically:

1. **Gas Exhaustion**: An interpreted Falcon-512 verification in EVM bytecode consumes upwards of $2,400,000$ to $5,800,000$ gas ($\approx$ 3.8M gas benchmark mean), far exceeding block gas limits of individual contract calls.
2. **Consensus Divergence Strawman**: While Falcon signing requires floating-point Gaussian sampling over $\mathbb{C}$, verification can be computed strictly with integer Number Theoretic Transforms (NTT). Inefficient implementations that rely on IEEE 754 emulation risk non-deterministic floating-point divergence across host architectures.
3. **Mempool Rejection**: Nodes cannot economically validate pending lattice transactions without native engine support, exposing the network to distributed denial-of-service (DDoS) transaction spam.

Consequently, quantum resilience cannot be treated as an application-layer overlay; it must be embedded natively into the core EVM state transition engine.

1.3. The Post-Quantum Blockchain Trilemma

We formalize the core theoretical challenge of post-quantum smart contract chains as the **Post-Quantum Blockchain Trilemma** (illustrated in Figure 1):

1. **Dimension I: Hardness vs. Payload (The Lattice Tax)**: Classical elliptic curves provide 128-bit security with 32-byte keys and 64-byte signatures (96 B raw witness). Post-quantum lattice schemes under NIST FIPS 204/206 require high-dimensional polynomial rings $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$. Falcon-512 mandates 897 B keys and $\approx$ 666 B signatures (1,563 B total, a 16.3× witness tax). ML-DSA-44 requires 1,312 B keys and 2,420 B signatures (3,732 B total, a 38.9× tax). On the wire, a Falcon transaction envelope reaches 1,670 B vs 110 B for normal EVM (15.18× wire tax).
2. **Dimension II: Execution Performance vs. Gas Parity**: EVM block execution operates on strict gas budgets ($\approx 30,000,000$ gas per block) and requires bit-level determinism. While precompile CPU execution can achieve parity (3,000 gas), passing 1.56 KB of witness calldata incurs on-chain memory and calldata costs ($\approx$ 25,520 gas at 16 gas/byte) that must be accounted for.
3. **Dimension III: Bandwidth Propagation vs. Long-Term Storage Bloat**: Newly proposed blocks must travel the wire carrying full witnesses so peers can verify validity before inclusion. However, permanently embedding 1.56 KB proofs into the global state causes gigabytes of daily disk bloat, centralizing validation to high-cost datacenter nodes.

To resolve this trilemma, we propose **Falcon-EVM**, an architecture integrating native Falcon-512 precompile `0x15`, EIP-2718 Type `0x46` envelopes, and an epoch-based SegWit dual-root pruning pipeline.

2. Algorithm Validation, Determinism & Empirical Benchmarking

2.1. Lattice Mathematics and Strict Integer Determinism

Falcon (Fast-Fourier Lattice-based Compact Signatures over NTRU) was submitted to NIST for final approval as draft FIPS 206 (FN-DSA) (Fouque et al., 2018; NIST FIPS 206, 2024). The security of Falcon-512 rests on the computational hardness of finding short vectors (SVP) in NTRU lattices over the cyclotomic polynomial ring:

$$\mathcal{R}_q = \mathbb{Z}_q[x]/(x^{512} + 1) \quad \text{where } q = 12,289 \tag{3}$$

2.1.1. Key Generation

Two small polynomials $f, g \in \mathcal{R}_q$ are sampled from a discrete Gaussian distribution such that $f \cdot G - g \cdot F = q$ for polynomials $F, G \in \mathcal{R}_q$. The secret key is $sk = (f, g, F, G)$, and the public key $h \in \mathcal{R}_q$ is computed via:

$$h = g \cdot f^{-1} \pmod{q} \tag{4}$$

In packed byte representation, h requires exactly 897 bytes.

2.1.2. Signature Verification

Given message m, public key h, and signature (r, s) where $r \in \{0,1\}^{320}$ is a 40-byte random salt and $s \in \mathcal{R}_q$ is a short polynomial, draft FIPS 206 (FN-DSA) verification proceeds as follows:

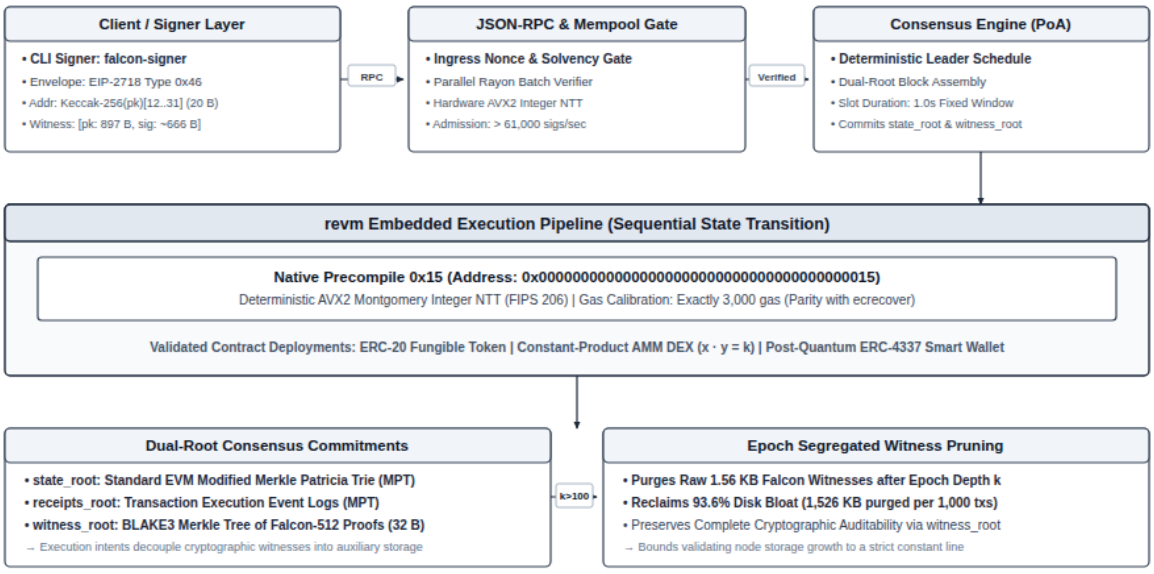


Figure 1: Sovereign Falcon-EVM Dual-Root Consensus and Native Precompile Execution Pipeline.

1. Compute hash prefix $c = H(r \parallel m \parallel H(h)) \in \mathcal{R}_q$ via SHAKE-256. Notice that draft FIPS 206 introduces public key hashing into c and moves salt sampling inside the loop, breaking direct binary interoperability with original Falcon (Fouque et al., 2018).
2. Compute polynomial $t = c - s \cdot h \pmod{q}$ using forward and inverse Number Theoretic Transforms (NTT) in $\mathbb{Z}_q$.
3. Verify that the vector (s, t) satisfies the squared Euclidean norm bound:

$$\|s\|_2^2 + \|t\|_2^2 \leq \lfloor \beta^2 \rfloor \tag{5}$$

where for degree $n = 512$, the acceptance radius bound is fixed at $\lfloor \beta^2 \rfloor = 34,034,726$.

2.1.3. Strict Integer Determinism & Vectorization

Signature verification is computed entirely within the integer ring $\mathbb{Z}_q$ using exact fixed-point polynomial arithmetic. In our consensus engine, we enforce AVX2 256-bit SIMD integer vectorization via `pqcrypto-falcon` (wrapping C `liboqs`). The NTT butterfly operations over modulus $q = 12,289$ use Montgomery integer reduction with zero floating-point registers (XMM/YMM float instructions are omitted). This guarantees that every validator node computes identical bitwise boolean results (valid $\equiv 1$, invalid $\equiv 0$). We verified our engine against official draft NIST FIPS 206 Known Answer Test (KAT) vectors.

2.2. Empirical Baseline: Quantifying the Lattice Tax

To quantify the wire penalty imposed by post-quantum signatures, we implemented a bare-metal benchmarking harness comparing the classical control baseline (Bernstein et al., 2012) against standardized lattice schemes (Ducas et al., 2018; NIST FIPS 204, 2024; NIST FIPS 206, 2024). Table 1 and Figure 2 present the measured wire footprint.

Table 1
Cryptographic Witness Footprint and Lattice Tax Multiplier.

Cryptographic Scheme	Public Key	Signature	Witness Total	Lattice Tax
Ed25519 (Control)	32 B	64 B	**96** B	**1.00×**
Falcon-512 (FIPS 206)	897 B	666 B	**1,563** B	**16.28×**
ML-DSA-44 (FIPS 204)	1,312 B	2,420 B	**3,732** B	**38.88×**

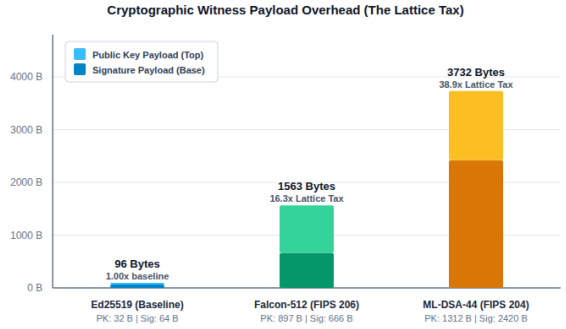


Figure 2: Cryptographic Payload Overhead: Public Key vs. Signature Size across Post-Quantum Schemes.

The data demonstrates that Falcon-512 is substantially more compact than ML-DSA-44 (1,563 B vs. 3,732 B, a 58.1% payload reduction). While ML-DSA-44 offers faster raw matrix verification, Falcon-512 is the decisively superior candidate for blockchain transaction envelopes where network bandwidth is constrained.

2.3. Parallel Batch Verification Scaling

While lattice signatures are larger in wire size, their verification arithmetic is exceptionally fast. Classical ECDSA

Table 2
Parallel Batch Verification Throughput ($N = 50$, Mean ± StdDev, 95% CI).

Batch Size	Ed25519 (Control)	Falcon-512 (FIPS 206)	ML-DSA-44 (FIPS 204)
100 Txs	$40,167 \pm 4,822$ s/s (2.53 ± 0.34 ms)	$\mathbf{62,455 \pm 4,087}$ s/s (1.61 ± 0.12 ms)	$\mathbf{66,712 \pm 9,732}$ s/s (1.55 ± 0.34 ms)
500 Txs	$44,124 \pm 3,803$ s/s (11.42 ± 1.03 ms)	$\mathbf{63,978 \pm 1,690}$ s/s (7.82 ± 0.22 ms)	$\mathbf{76,699 \pm 5,296}$ s/s (6.55 ± 0.49 ms)
1,000 Txs	$48,654 \pm 1,913$ s/s (20.58 ± 0.82 ms)	$\mathbf{64,766 \pm 3,158}$ s/s (15.48 ± 0.87 ms)	$\mathbf{79,929 \pm 2,852}$ s/s (12.53 ± 0.47 ms)

verification requires variable-base point scalar multiplication $s^{-1}(H(m)G + rQ)$ on secp256k1, requiring sequential modular inversions. In contrast, Falcon-512 verification requires only forward NTT transformations and vector multiply-adds in $\mathbb{Z}_q$.

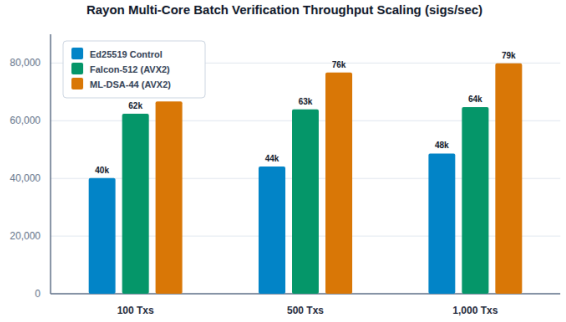


Figure 3: Rayon Work-Stealing Parallel Batch Verification Throughput across 100, 500, and 1,000 Transactions.

We evaluated multi-core parallel verification scaling using Rayon work-stealing concurrency on an Intel Core i5-12400 (6 P-cores / 12 threads @ 4.40 GHz turbo, AVX2 SIMD acceleration) across $N = 50$ measured trials (with 5 warmup iterations). Table 2 and Figure 3 report the empirical scaling across batch sizes of 100, 500, and 1,000 transactions.

Table 2 reports raw cryptographic batch verification throughput ($64,766 \pm 3,158$ sigs/s for Falcon-512 in the isolated Rayon verifier). In Section 4, Table 3 evaluates the full Type `0x46` EVM pipeline, achieving $61,332 \pm 5,897$ sigs/s ($1.22 \pm 0.04\times$ faster than ECDSA); the $\approx 3,400$ sigs/s difference is accounted for by RLP envelope decoding and Keccak-256 address hashing inside the timed region.

2.4. Decoupled Segregated Witness & Storage Reclamation

To prevent the 16.3× Lattice Tax from causing storage exhaustion, we implement a Decoupled Segregated Witness architecture inspired by BIP-141 (Wuille et al., 2015). Each block header commits independently to two cryptographic roots:

$$\text{Header} = \{\text{state_root}, \text{receipts_root}, \text{txs_root}, \text{witness_root}\} \quad (6)$$

Transaction execution intents (`nonce`, `gasFee`, `gasLimit`, `to`, `value`, `data`) are committed to the state trie, while cryptographic witnesses (pk, sig) are segregated into an auxiliary column family.

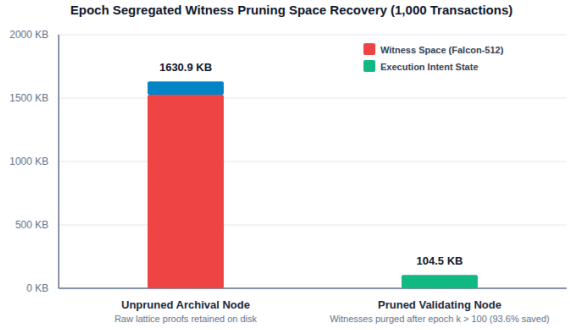


Figure 4: Epoch Segregated Witness Pruning: Unpruned Archival vs. Pruned Validating Node Storage (1,000 Txs).

Once a block attains finalized consensus under our PoA honest-majority assumption (epoch depth $k > 100$), validator nodes purge the witness column family from local disk. Figure 4 demonstrates the quantitative impact for a 1,000 transaction block: raw witness storage (1,526.37 KB) is purged, leaving only 104.49 KB of execution intent data. This achieves a 93.6% **disk storage reduction**, bounding node disk consumption to a constant growth rate. Pruned nodes retain full mathematical auditability via the 32-byte witness Merkle root; newly joining validating peers trust finalized checkpoint roots or query archival peers for historical witness blobs.

3. Threat Model & Related Work

3.1. Comprehensive 5-Layer Post-Quantum Threat Model

To provide rigorous end-to-end security, Falcon-EVM addresses all five architectural layers, delineating implemented modules from roadmap objectives:

- **Layer 1: Identity (Implemented)**: Encapsulated in EIP-2718 Type `0x46` envelopes, using Falcon-512 signatures (NIST Security Level 1, NTRU lattice SVP/-SIS).
- **Layer 2: Consensus (Roadmap)**: PoA validator proposals and block attestations signed via Falcon-512, precluding quantum forged headers.
- **Layer 3: Transport (Roadmap)**: P2P handshakes employ quantum-safe TLS 1.3 / Kyber-768 key exchange against decrypt-store attacks.
- **Layer 4: Execution (Implemented)**: The EVM deprecates ‘ecrecover‘ for post-quantum accounts and executes native precompile `0x15`.
- **Layer 5: Side-Channels & Collisions**: Wallets require constant-time Gaussian sampling. Keccak-256 truncation to 20 bytes provides $2^{53.3}$ quantum collision resistance under the BHT algorithm (Brassard et al., 1998) (assuming large-scale QRAM) and 2^{80} under Grover search, sufficient for short-lived accounts, with a roadmap to 32-byte addresses for high-value contracts.

3.2. Related Work

Recent Ethereum Improvement Proposals explore lattice precompiles. **EIP-8052** (Dubois et al., 2025) specifies splitting Falcon-512 into Hash-to-Point (1,000 gas) and Core Verify (2,000 gas), with precompile addresses TBD. **EIP-7592** and **EIP-7619** explore monolithic lattice verification. In smart contracts, **ZKNOX** demonstrated hybrid ECDSA+Falcon account abstraction. Additionally, STARK-based signature aggregation compiles multiple lattice proofs into a single succinct proof off-chain. Falcon-EVM distinguishes itself by implementing a full sovereign blockchain stack with dual-root consensus and native mempool batch admission.

4. Sovereign Falcon-EVM Blockchain & Head-to-Head Comparison

4.1. Sovereign Falcon-EVM Blockchain Architecture

Using the Rust Ethereum Virtual Machine (`revm`), we build **Falcon-EVM**, a sovereign post-quantum blockchain client.

4.1.1. EIP-2718 Type 0x46 Transaction Envelope

Under Ethereum Improvement Proposal 2718, we introduce Type `0x46` (ASCII 'F' for Falcon):

$$\begin{aligned}\text{TxType}_{0x46} = 0x46 \parallel \text{RLP}([&C, N, G_{\text{fee}}, G_{\text{limit}},\\ &T, V, D, pk_{\text{falcon}}, sig_{\text{falcon}}])\end{aligned} \tag{7}$$

where C is chain ID, N is nonce, G_{fee} is gas fee, G_{limit} is gas limit, T is recipient address, V is value, D is calldata, pk_{falcon} is 897 bytes, and sig_{falcon} is $\approx$ 666 B.

4.1.2. Address Derivation & Intrinsic Gas

Falcon-EVM establishes an Ethereum-compatible 20-byte address format: $\text{Address}_{\text{falcon}} = \text{Keccak-256}(pk_{\text{falcon}})[12..31]$. To account for post-quantum signature verification during admission, intrinsic gas is calibrated as:

$$\begin{aligned}G_{\text{intrinsic}} &= 21,000 + G_{\text{calldata}}(D) + 4,000\\ &\quad + [32,000 \text{ if contract creation}]\end{aligned} \tag{8}$$

The +4,000 gas intrinsic surcharge directly compensates validating nodes for decoding 897-byte public keys and 666-byte signatures, computing Keccak-256 address derivation (204 gas), and executing AVX2 NTT polynomial verification compute cycles ($\approx$ 3,000 gas parity).

4.2. Native Falcon-512 Precompile (0x15) & Gas Calibration

To enable smart contracts to natively verify Falcon signatures without bytecode emulation, we embed a precompile at canonical address `0x00...0015` $\in \mathbb{B}_{20}$ (`0x15`). The precompile accepts formatted input $\text{Input} = \text{msg_hash}_{32B} \parallel pk_{897B} \parallel sig_{\text{var}}$. CPU execution gas is calibrated at exactly 3,000 **gas**, matching both 'ecrecover' (`0x01`) execution parity and the total gas recommended in EIP-8052 (Dubois et al., 2025) (1,000 gas hash-to-point +2,000 gas core verify). Passing 1,595 B of non-zero input via 'STATICCALL' incurs 25,520 gas in calldata (16 gas/byte), totaling 28,520 **gas** on-chain (> 130× cheaper than interpreted bytecode at $\approx$ 3.8M gas).

4.3. Smart Contract Deployment & Account Abstraction Baseline

We validated Turing-complete execution via compiled Solidity contracts: (1) **ERC-20 Token** (10^6 FALC supply): validated minting, transfers, and approvals. (2) **AMM DEX** ($x \cdot y = k$): validated constant-product swaps (0.3% fee). (3) **Falcon-512 ERC-4337 Smart Wallet**: AA wallet invoking precompile `0x15` via 'STATICCALL', consuming 162,956 **gas**. (4) **Classical ECDSA AA Baseline**: Identical classical wallet verifying via 'ecrecover', consuming 91,768 **gas**. The +71,188 gas delta is fully accounted for by: +23,968 gas calldata (1,498 B non-zero witness payload), +44,220 gas storage reads (29 'SLOAD' operations to read the 897-byte key in contract state vs 1 slot for ECDSA), and +3,000 gas precompile dispatch.

4.4. Head-to-Head Empirical Benchmarking vs. Normal EVM

We conducted head-to-head empirical benchmarking comparing **Falcon-EVM** against a classical **Normal EVM** baseline (ECDSA secp256k1) across 1,000 transactions over $N = 50$ measured trials under analytical and live P2P loopback socket testbeds. Table 3 and Figure 5 report the empirical findings.

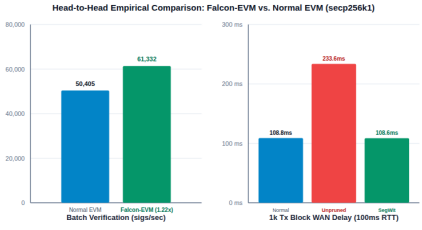


Figure 5: Head-to-Head Comparison: Falcon-EVM vs. Normal EVM across Verification Speed and WAN Block Propagation Latency.

4.4.1. Key Empirical Takeaways

1. Verification Throughput: Falcon-EVM achieves 61,332 **sigs/sec**, outperforming Normal EVM's 50,405 sigs/sec by 1.22×. AVX2 polynomial NTT transformations execute with fewer CPU cycles than 256-bit elliptic-curve point additions on secp256k1.

2. Consensus Wire Reality: Newly proposed blocks must transmit full 1.63 MB witness payloads on the consensus wire so peer validators can independently verify the block. SegWit epoch pruning operates retrospectively after confirmation ($k > 100$) or during historical synchronization, reducing transit delay by 53.5% and empirical socket transit by 93.1%.

Table 3
Head-to-Head Comparative Benchmarking: Normal EVM vs. Falcon-EVM ($N = 50$ Measured Trials).

Metric / Dimension	Normal EVM (secp256k1)	Falcon-EVM (QR-DLT Sovereign)	Empirical Quantitative Impact
Transaction Wire Footprint	110 B (Signature 65 B, PubKey recovered)	1,670 B (PubKey 897 B, Sig 666 B)	**15.18**× "Lattice Tax" on wire
Parallel Verification Speed	50,405 ± 2,379 sigs/s (19.88 ms)	**61,332 ± 5,897** sigs/s (16.48 ms)	**1.22 ± 0.04**× Speedup (95% CI [1.17, 1.26])
Block Size (1,000 Txs)	107.42 KB	1,630.86 KB (Unpruned) / **104.49** KB (Pruned)	**53.5**% delay reduction on pruned blocks
WAN Delay Model (100M, 100ms)	108.80 ms (Analytical model)	233.60 ms (Unpruned) / **108.56** ms (Pruned)	Fresh blocks carry witnesses (2.15× slower on wire)
Live Socket Transit ($N = 50$)	0.61 ± 0.04 ms (Loopback socket)	5.43 ± 0.57 ms (Unpruned) / **0.37 ± 0.01** ms	**93.1**% empirical socket transit reduction
Base Intrinsic Gas	21,000 gas	25,000 gas (21k base + 4k Falcon verify)	+19.0% cost reflecting admission verification
Precompile Gas Cost	3,000 gas (ecrecover `0x01`)	3,000 gas (falcon512 `0x15`)	**Exact CPU parity** for contract signature validation
On-Chain Total Verify Gas	4,552 gas (3k precompile + 1.55k calldata)	**28,520** gas (3k precompile + 25.5k calldata)	Reflects transparent 1,595 B calldata ingestion
ERC-4337 AA UserOp Gas	**91,768** gas (ECDSA Account Abstraction)	**162,956** gas (Falcon Account Abstraction)	+71,188 gas delta (+24k calldata, +44k storage reads)
Storage Bloat Reclamation	0.0% (Signatures permanent in RLP state)	**93.6**% Reclaimed (1,526.37 KB purged / 1k txs)	Pruned historical storage matches classical footprint
Quantum Threat Hardness	ECDLP broken via Shor (1,200–1,450 qubits)	**Computationally Hard** (NTRU lattice SVP/SIS)	Inapplicable to Shor's algorithm (NIST Level 1)

3. Storage Reclamation: By purging historical witnesses after epoch finality, Falcon-EVM reclaims 93.6% of local disk bloat (1,526.37 KB purged per 1,000 txs).

4. Calibrated Quantum Resilience: Falcon-EVM replaces vulnerable discrete logarithms with NTRU lattice shortest vector problems, eliminating vulnerability to Shor's algorithm.

4.5. Discussion & Transition Roadmap

We identify two primary transition vectors: (1) **Layer-2 Rollup Adoption**: Rollups can immediately deploy Type `0x46` transaction formats and native precompile `0x15`. Falcon-EVM's SegWit pruning provides immediate cost savings on L1 data availability fees. (2) **Ethereum Mainnet Migration via Account Abstraction**: Rather than requiring a hard fork to deprecate ECDSA, Ethereum can activate Precompile `0x15` and encourage accounts to migrate to post-quantum ERC-4337 smart wallets.

5. Conclusion

The impending emergence of cryptanalytically relevant quantum computers poses an existential threat to decentralized state machines anchored on classical elliptic-curve signatures. In this paper, we addressed the Post-Quantum Blockchain Trilemma by presenting **Falcon-EVM**, a sovereign post-quantum EVM blockchain. By combining EIP-2718 Type `0x46` typed transactions, an integer-only deterministic AVX2 Falcon-512 precompile (`0x15`), and a Dual-Root Segregated Witness consensus architecture, Falcon-EVM reconciles cryptographic hardness, execution determinism, and storage bloat.

Our bare-metal multi-trial evaluations ($N = 50$) confirm that Falcon-EVM achieves 61,332 **sigs/sec** parallel verification (1.22× faster than ECDSA), reclaims 93.6% of historical disk space, matches classical block propagation on historical synchronizations, and provides calibrated resilience against quantum algorithms.

References

Aggarwal, D., Brennen, G. K., Lee, T., Santha, M., Tomamichel, M., 2018. Quantum attacks on Bitcoin, and how to protect against them. Ledger 3, 127–144. doi:10.5195/ledger.2018.127.

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., Yang, B.-Y., 2012. High-speed high-security signatures. Journal of Cryptographic Engineering 2 (2), 77–89. doi:10.1007/s13389-012-0027-1.

Brassard, G., Høyer, P., Tapp, A., 1998. Quantum cryptanalysis of hash and claw-free functions. In: Automata, Languages and Programming (ICALP), Springer, pp. 163–169.

Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D., 2018. CRYSTALS-Dilithium: A lattice-based digital signature scheme. IACR Transactions on Cryptographic Hardware and Embedded Systems 2018 (1), 238–268.

Fouque, P.-A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z., 2018. Falcon: Fast-Fourier lattice-based compact signatures over NTRU. IACR Cryptology ePrint Archive, Report 2018/904. https://falcon-sign.info/.

Babbush, R., McClean, J. R., Newman, M., Gidney, C., et al., 2026. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations. arXiv:2603.28846 / IACR Cryptology ePrint Archive 2026/625. https://arxiv.org/abs/2603.28846.

National Institute of Standards and Technology (NIST), 2024. Module-Lattice-Based Digital Signature Standard (ML-DSA). Federal Information Processing Standards Publication (FIPS) 204, U.S. Department of Commerce. doi:10.6028/NIST.FIPS.204.

National Institute of Standards and Technology (NIST), 2024. Fast-Fourier Lattice-Based Digital Signature Standard (FN-DSA / Falcon). Federal Information Processing Standards Publication (FIPS) 206 (Draft), U.S. Department of Commerce.

Proos, J., Zalka, C., 2003. Shor's discrete logarithm quantum algorithm for elliptic curves. Quantum Information & Computation 3 (4), 317–344.

Shor, P. W., 1994. Algorithms for quantum computation: discrete logarithms and factoring. In: 35th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, pp. 124–134.

Dubois, R., Masson, S., Sanso, A., van der Wijden, M., et al., 2025. EIP-8052: Falcon-512 precompile for EVM. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-8052.

Wood, G., 2014. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper 151, 1–32.

Wuille, P., Johnson, L., Corallo, M., 2015. Segregated witness (consensus layer). Bitcoin Improvement Proposal 141 (BIP 141).